1. Introduction

Servlet is a Java programming language class, part of Java Enterprise Edition (Java EE). Sun Microsystems developed its first version 1.0 in the year 1997. Its current Version is Servlet 3.1.

Servlets are used for creating dynamic web applications in java by extending the capability of a server. It can run on any web server integrated with a Servlet container.

1.1 Java Servlet Process

The process of a servlet is shown below:

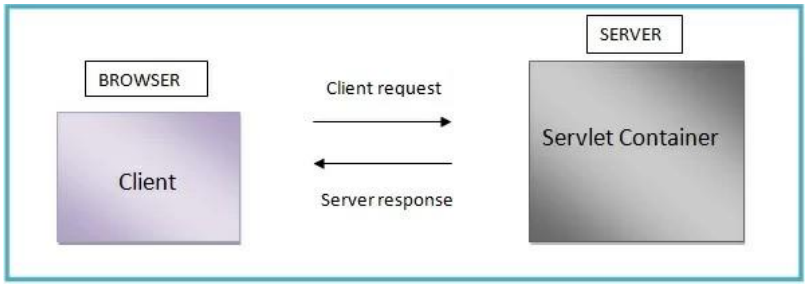

Figure 1: servlet processing of user requests

- A Request is sent by a client to a servlet container. The container acts as a Web server.
- The Web server searches for the servlet and initiates it.
- The client request is processed by the servlet and it sends the response back to the server.
- The Server response is then forwarded to the client.

1.2 Merits

- Servlets are platform independent as they can run on any platform.
- The Servlet API inherits all the features of the Java platform.
- It builds and modifies the security logic for server-side extensions.
- Servlets inherit the security provided by the Web Server.
- In Servlet, only a single instance of the requests runs concurrently. It does not run in a separate process. So, it saves the memory by removing the overhead of creating a new process for each request.

2. Life Cycle

Servlet lifecycle describes how the servlet container manages the servlet object.

- Load Servlet Class
- Servlet Instance is created by the web container when the servlet class is loaded
- `init()`

:This is called only once when the servlet is created. There is no need to call it again and again for multiple requests.

```
public void init() throws ServletException {

 }
```

- `service()`

  : It is called by the web container to handle request from clients. Here the actual functioning of the code is done. The web container calls this method each time when request for the servlet is received.

It calls

`doGet()`

,

`doPost()`

,

`doTrace()`

,

`doPut()`

,

`doDelete()`

and other methods

- `doGet()`

  :

```
public void doGet(HttpServletRequest request,HttpServletResponse response)
    throws ServletException, IOException {
// code
}
```

- `doPost()`

  :

```
public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
// code4
}
```

- `destroy()`

  : It is used to clean resources and called before removing the servlet instance.

```
public void destroy()
```

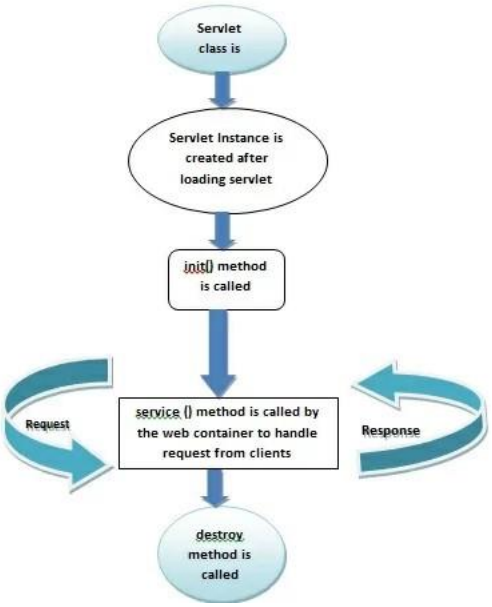

Figure 2: Servlet Life Cycle

3. Container

It is known as servlet engine which manages Java Servlet components on top of a web server to the request send by the client.

3.1 Services

Servlet Container provides the following services:

- It manages the servlet life cycle.
- The resources like servlets, JSP pages and HTML files are managed by servlet container.
- It appends session ID to the URL path to maintain session.
- Provides security service.
- It loads a servlet class from network services, file systems like remote file system and local file system.

3.2 Servlet Container Configurations

The servlet container can be configured with the web server to manage servlets in three ways listed below:

- Standalone container
- In-process container
- Out-process container

Standalone container: In this type the Web Server functionality is taken by the Servlet container. Here, the container is strongly coupled with the Web server.

In-Process container: In this the container runs within the Web server process.

Out-Process container: In this type there is a need to configure the servlet container to run outside the Web server process. It is used in some cases like if there is a need to run Servlets and Servlet container in different process/systems.

4. Demo: To start with

Here is an example showing Demo Servlet. Follow these steps to start with your first Servlet Application in NetBeansIDE.

Step 1: Open NetBeansIDE -> File -> New Project->WebApplication -> Set Project name as WebApplicationServletDemo

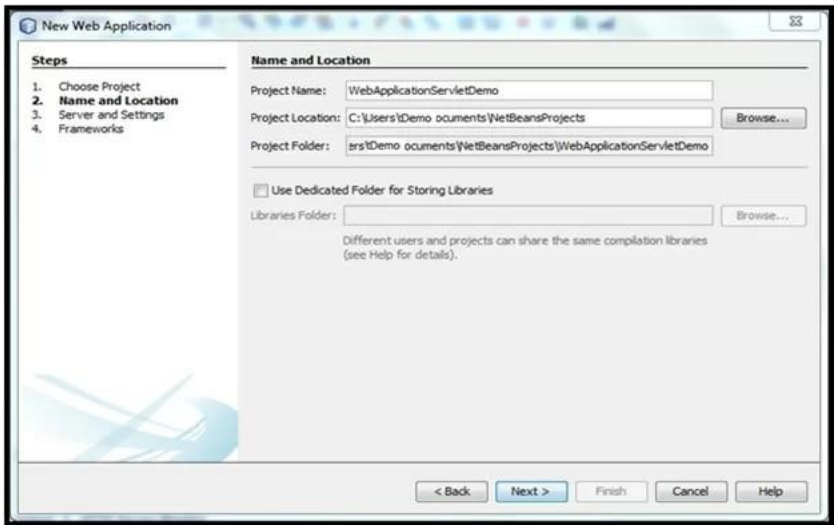

Figure 3: Create new WebApplication project in NetBeansIDE: WebApplicationServletDemo

Step 2: Now click on Next >as shown above. This will create new project with the following directory structure.

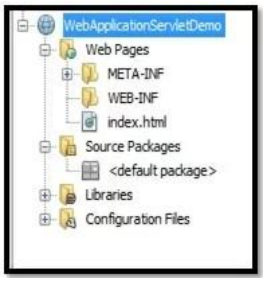

Figure 4: Project Directory after creating new project

Step 3: Create new servlet application by Right Clicking on Project Directory-> New -> Servlet

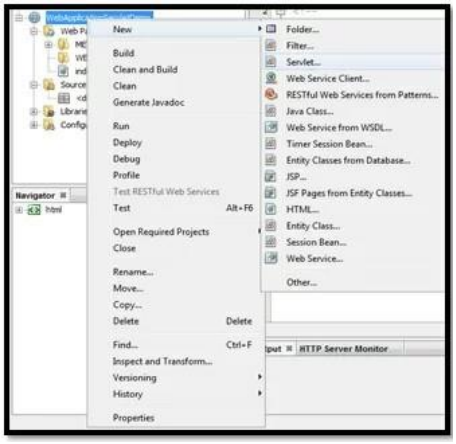

Figure 5: Adding Servlet file

Step 4: Add the Servlet Class Name as "ServletDemo" and click on Next.

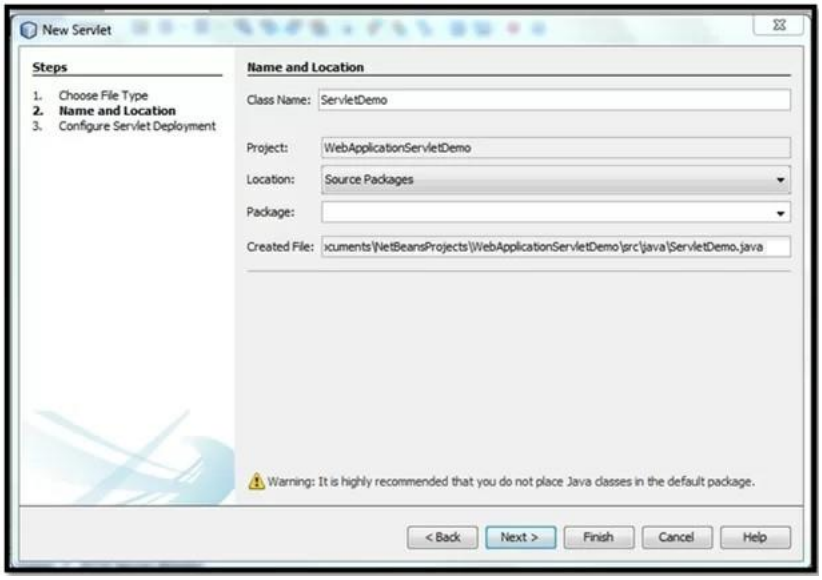

Figure 6: Adding Servlet Class Name

Step 5: Now, Configure Servlet Deployment by checking "Add information to deployment descriptor (web.xml)" and adding URL Pattern (the link visible) as ServletDemo. This step will generate web.xml file in WEB-INF folder.

Figure 7: Configuring Servlet Deployment

Step 6: Click on Finish as shown above, this will add ServletDemo.java servlet under project directory. Check the changes under Directory Structure:

Figure 8: Changes under project directory after configuring

Here is the code for deployment descriptor (web.xml) with URL-patter as /ServletDemo:

Listing 1: web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchem
<servlet>
<servlet-name>ServletDemo</servlet-name>
<servlet-class>ServletDemo</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ServletDemo</servlet-name>
    <url-pattern>/ServletDemo</url-pattern>
```

```
</servlet-mapping>
<session-config>
<session-timeout>
30
</session-timeout>
</session-config>
</web-app>
```

Here,

```
<servlet-name>: name given to Servlet
<servlet-class>: servlet class
<servlet-mapping>: maps internal name to URL
<url-pattern>: link displays when Servlet runs
```

The hyperlink Next is mentioned as ServletDemo. So, when the user will click on it, the page will redirect to ServletDemo servlet whose url-pattern is mentioned as ServetDemo:

Listing 2: index.html

```
<html>
<head>
<title>Welcome</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<div><h2>Welcome</h2></div>
<p>We're still under development stage. Stay Tuned for our website's new design and learning content.</p>
<a href="ServletDemo"><b>Next</b></a>
</body>
</html>
```

Listing 3: ServletDemo.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class ServletDemo extends HttpServlet {

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet ServletDemo</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet ServletDemo at " + request.getContextPath() + "</h1>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try {
            /* TODO output your page here. You may use following sample code. */
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlets</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<br /><p><h2>First Demo Servlet application</h2><br />Here, the URL-pattern is Serv
            out.println("<br /><br /><a href=\"index.html\">Previous Page</a>");
            out.println("</body>");
            out.println("</html>");
        }
        finally
        {
        out.close();
        }
    }
}
```

Figure 9: Output showing index.html welcome page

Figure 10: Output showing redirection to ServletDemo.java

5. Filter

Filters transform the content of requests, responses, and header information from one format to another. These are reusable codes.

- Filter class is declared in the deployment descriptor.
- It is used to write reusable components.
- The request is process before it is called using filters.
- It can be used under a web application for some tasks like:
  - Validation
  - Compression
  - Verification
  - Internationalization

5.1 Interface

It consists of these 3 filters:

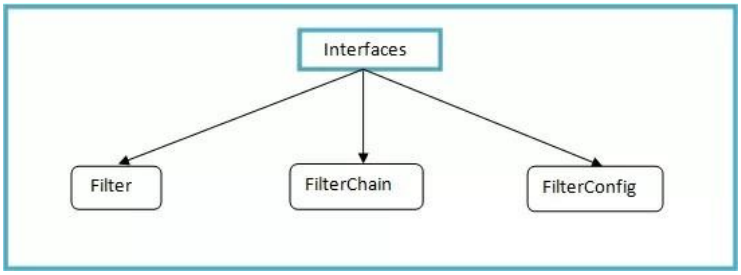

Figure 11: Filter API Interfaces

Filter

This is the initial and basic interface which all filter class should implement.

```
Java.servlet.Filter
```

interface has the following methods:

Methods	Description
`init(FilterConfig)`	This method initializes a filter
`doFilter(ServletRequest, ServletResponse, FilterChain)`	This method encapsulates the service logic on ServletRequest to generate ServletResponse. FilterChain is to forward request/response pair to the next filter.
`destroy()`	It destroys the instance of the filter class.

FilterConfig

Its object is used when the filters are initialized. Deployment descriptor (web.xml) consists of configuration information. The object of FilterConfig interface is used to fetch configuration information about filter specified in web.xml. Its methods are mentioned below:

Methods	Description

`getFilterName()`	It returns the name of filter in web.xml
`getInitParameter(String)`	It returns specified initialization parameter's value from web.xml
`getInitParameterNames()`	It returns enumeration of all initialization parameters of filter.
`getServletContext()`	It returns ServletContext object.

FilterChain

It stores information about more than 1 filter (chain). All filters in this chain should be applied on request before processing of a request.

5.2 Example

This is an example showing filters application in NetBeansIDE. Create a WebApplication project WebApplicationFilterDemo in the same ways as shown under Demo section. New Filter can be added in the web application by Right Clicking on Project Directory -> New -> Filter

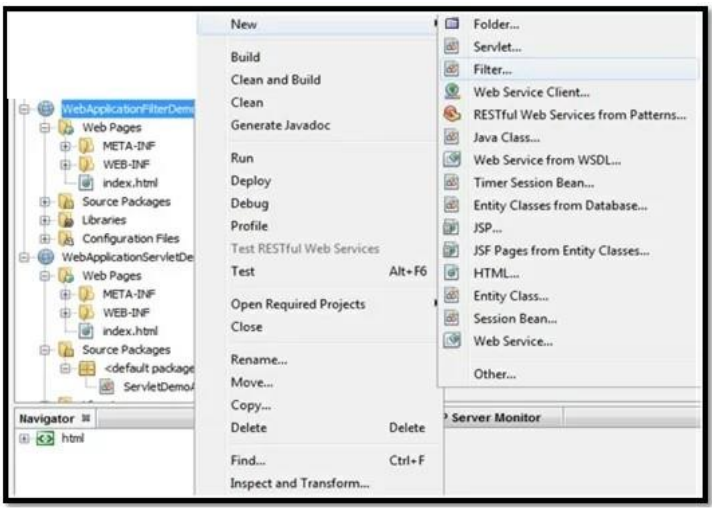

Figure 12: Add new Filter to web application

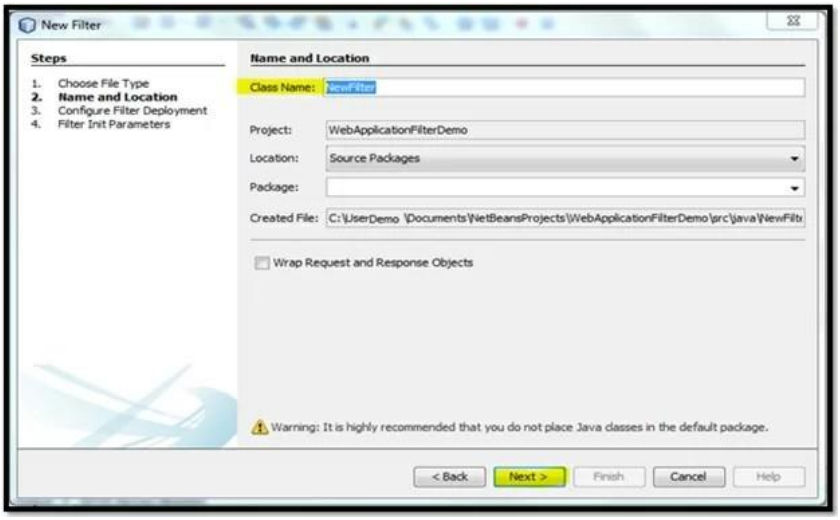

Figure 13: Add Class Name as NewFilter and click on Next

Configure Filter Deployment by checking "Add information to deployment descriptor (web.xml)". Now, the Next button is disabled here due to an error highlighted in Figure 13. The error "Enter at least one URL pattern" can be solved by clicking on "New".

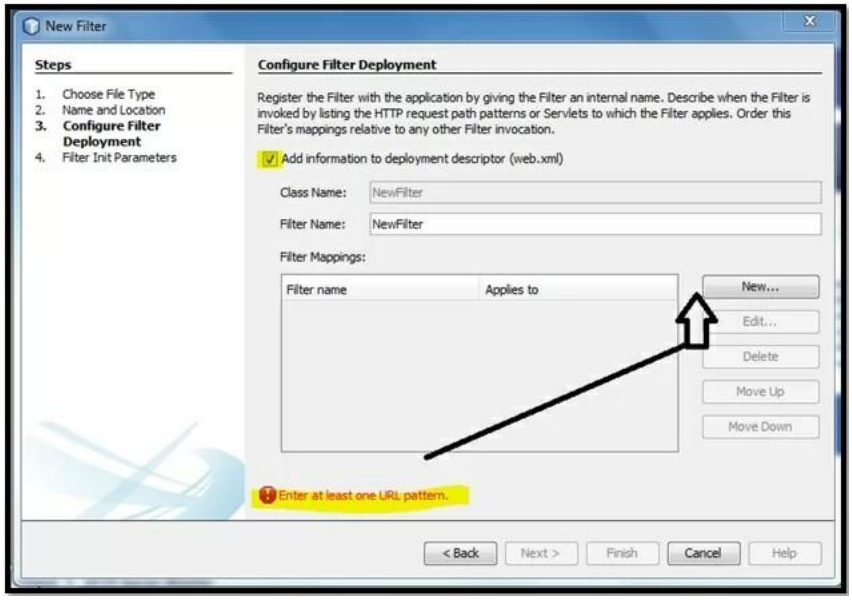

Figure 14: Configure Filter Deployment by checking "Add information to deployment descriptor(web.xml)"

Now, filter is mapped by adding URL-pattern as shown in Figure 15.

Figure 15: Filter mapping

After adding new filter and clicking on OK, the error will get resolved. Now, add init-parameter with name and value. Then click Finish.

Figure 16: Adding init-parameter

Listing 4: web.xml

The Filter NewFilter can be applied to every servlet as /* is specified here for URL-pattern.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchem
<filter>
<filter-name>NewFilter</filter-name>
<filter-class>NewFilter</filter-class>
  <init-param>
      <param-name>newParam</param-name>
      <param-value>valueOne</param-value>
  </init-param>
</filter>
<filter-mapping>
     <filter-name>NewFilter</filter-name>
     <url-pattern>/*</url-pattern>
</filter-mapping>
<session-config>
    <session-timeout>
    30
   </session-timeout>
</session-config>
</web-app>
```

Listing 5: NewFilter.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

public class NewFilter implements Filter {

    public void init(FilterConfigfilterConfig) {
        // init parameter
        String value = filterConfig.getInitParameter("newParam");

        // displaying init parameter value
    System.out.println("The Parameter value: " + value);14
    }

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        // IP address of the client machine.
        String remoteAddress = request.getRemoteAddr();

        // Returns the remote address
        System.out.println("Remote Internet Protocl Address: "+ remoteAddress);

        chain.doFilter(request,response);
    }

    public void destroy( ){

    }
}
```

```
13-Nov-2014 14:43:16.441 INFO [http-nio-8084-exec-48] org.apache.catalina.core.StandardC
ontext.reload Reloading Context with name [/WebApplicationFilterDemo] is completed
The Parameter value: valueOne
Remote Internet Protocl Address: 0:0:0:0:0:0:0:1
```

Figure 17: Showing console output

6. Session

It is a collection of HTTP requests between client and server. The session is destroyed when it expires and its resources are back to the servlet engine.

6.1 Session Handling

It is a means to keep track of session data. This represents the data transferred in a session. It is used when session data from one session may be required by a web server for completing tasks in same or different sessions. Session handling is also known assession tracking.

6.2 Mechanisms of Session Handling

There are four mechanisms for session handling:

URL rewriting: The session data required in the next request is appended to the URL path used by the client to make the next request.

- Query String: A string appended after the requested URI is query string. The string is appended with separator as '?' character.

Example 1):

```
http://localhost:8080/newproject/login?user=test&passwd=abcde
```

- Path Info: It is the part of the request URI. Session data can be added to the path info part of the request URI.

Example 2):

```
http://localhost:8080/newproject/myweb/login;user=test&passwd=abcde
```

Hidden form field: A type of HTML form field which remains hidden in the view. Some other form fields are: textbox, password etc. This approach can be used with form-based requests. It is just used for hiding user data from other different types of users.

Example 3:

```
<input type="hidden" username="name" value="nameOne"/>
```

Cookies: It is a file containing the information that is sent to a client by a server. Cookies are saved at the client side after being transmitted to clients (from server)through the HTTP response header.

Cookies are considered best when we want to reduce the network traffic. Its attributes are name, value, domain, version number, path, and comment. The package

```
javax.servlet.http
```

consists of a class names Cookie.

Some methods in

```
javax.servlet.http.Cookie
```

class are listed below:

- `setValue (String)`
- `getValue()`
- `getName()`
- `setComment(String)`
- `getComment()`
- `setVersion(String)`
- `getVersion()`
- `setDomain(String)`
- `setPath(String)`
- `getPath()`
- `setSecure(boolean)`
- `getSecure(boolean)`

HTTP session: It provides asession management service implemented through HttpSession object.

Some HttpSession object methods are listed here; this is referred from the official Oracle Documentation:

Method	Description
`public Object getAttribute(String name)`	It returns the object bound with the specified name in this session or null if no object is bound under the name.
`public Enumeration getAttributeNames()`	It returns Enumeration of String objects containing the names of all the objects bound to this session.
`public String getId()`	It returns a string containing the unique identifier assigned to this session.
`public long getCreationTime()`	It returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.

`public long getLastAccessedTime()`	It returns the last time the client sent a request associated with this session.
`public int getMaxInactiveInterval()`	It returns the maximum time interval, in seconds that the servlet container will keep this session open between client accesses.
`public void invalidate()`	It Invalidates this session then unbinds any objects bound to it.
`public boolean isNew()`	It returns true if the client does not yet know about the session or if the client chooses not to join the session.

6.3 Example

Session Information like session id, session creation time, last accessed time and others is printed under this example.

Listing 6: ServletSession.java

```
01  import java.io.IOException;
02  import java.io.PrintWriter;
03  import java.util.Date;
04  import javax.servlet.ServletException;
05  import javax.servlet.http.HttpServlet;
06  import javax.servlet.http.HttpServletRequest;
07  import javax.servlet.http.HttpServletResponse;
08  import javax.servlet.http.HttpSession;
09
10  public class ServletSession extends HttpServlet {
11
12  @Override
13   protected void doGet(HttpServletRequest request, HttpServletResponse response)
14              throws ServletException, IOException {
15             // session object creation
16             HttpSessionnewSession = request.getSession(true);
17        // Session creation time.
18        Date cTime = new Date(newSession.getCreationTime());
19        // The last time the client sent a request.
20        Date lTime = new Date( newSession.getLastAccessedTime());
21
22        /* sets the time, in seconds, between client requests before the servlet container
23        invalidates this session */
24        newSession.setMaxInactiveInterval(1 * 60 * 60);
25        String str = "Website | Session";
26
27        response.setContentType("text/html");
28        PrintWriter out = response.getWriter();
29
30        String document =
31        "<!doctype html public \"-//w3c//dtd html 4.0 " +
32        "transitional//en\">\n";
33        out.println(document +
34                  "<html>\n" +
35                  "<head><title>" + str + "</title></head>\n" +
36                  "<body bgcolor=\"#bbf5f0\">\n" +
37                   "<h2>Website: Displaying Session Information</h2>\n" +
38                  "<table border=\"2\">\n" +
39                  "<tr>\n" +
40                  "  <td>Unique identifier assigned to this session</td>\n" +
41                  "  <td>" + newSession.getId() + "</td>"
42                + "</tr>\n" +
43                  "<tr>\n" +
44                  "  <td>The time when this session was created</td>\n" +
45                  "  <td>" + cTime +
46                  "  </td>"
47                + "</tr>\n" +
48                  "<tr>\n" +
49                  "  <td>The last time the client sent a request associated with this session</td>\n"
50                + "  <td>" + lTime +
51                  "  </td>"
52                + "</tr>\n" +
53                  "</tr>\n" +
54                  "<tr>\n" +
55                  "  <td> the maximum time interval, in seconds that the servlet container will keep this sess
56                  "  <td>" + newSession.getMaxInactiveInterval() +
57                  "  </td>"
58                + "</tr>\n" +
59                        "</table>\n" +
60                  "</body></html>");
61    }
62  }
```

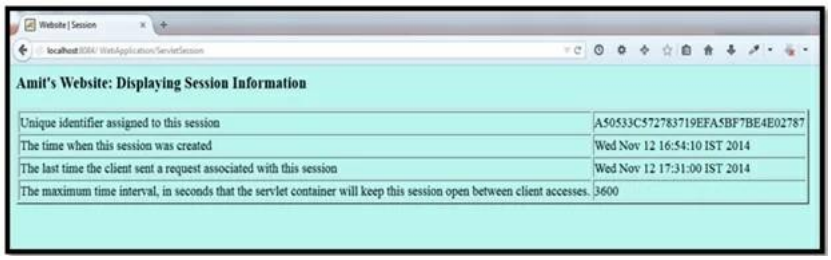

Figure 18: Displaying output

7. Exception Handling

Exceptions are used to handle errors. It is a reaction to unbearable conditions. Here comes the role of web.xml i.e. deployment description which is used to run JSP and servlet pages. The container searches the configurations in web.xml for a match. So, in web.xml use these exception-type elements for match with the thrown exception type when a servlet throws an exception.

7.1 Error Code Configuration

The /HandlerClass servlet gets called when an error with status code 403 occurs as shown below:

Listing 7: For Error code 403

```
<error-page>
  <error-code>403</error-code>
  <location>/HandlerClass</location>
</error-page>
```

7.2 Exception-Type Configuration

If the application throws IOException, then /HandlerClass servlet gets called by the container:

Listing 8: For Exception Type IOException

```
<error-page>
  <exception-type>java.io.IOException</exception-type >
  <location>/HandlerClass</location>
</error-page>
```

If you want to avoid the overhead of adding separate elements, then use

```
java.lang.Throwable
```

as exception-type:

Listing 9: For all exceptions mention java.lang.Throwable:

```
<error-page>
  <exception-type>java.lang.Throwable</exception-type >
  <location>/HandlerClass</location>
</error-page>
```

8. Debugging

Client-server interactions are in large number in Servlets. This makes errors difficult to locate. Different ways can be followed for location warnings and errors.

8.1 Message Logging

Logs are provided for getting information about warning and error messages. For this a standard logging method is used. Servlet API can generate this information using log() method. Using Apache Tomcat, these logs can be found in TomcatDirectory/logs.

8.2 Java Debugger

Servlets can be debugged using JDB Debugger i.e. Java Debugger. In this the program being debugged is sun.servlet.http.HttpServer. Set debugger's class path for finding the following classes:

- servlet.http.HttpServer
- server_root/servlets and server_root/classes: Through this the debugger sets breakpoints in a servlet.

8.3 Headers

Users should have some information related to structure of HTTP headers. Issues can be judged using them which can further locate some unknown errors. Information related to HTTP headers can help you in locating errors. Studying request and response can help in guessing what is not going well.

8.4 Refresh

Refresh your browser's web page to avoid it from caching previous request. At some stages, browser shows request performed previously. This is a known point but can be a problem for those who are working correctly but unable to display the result properly.

Listing 21: ServletDebugging.java

Here, Servlet Debugging is shown which displays the errors in Tomcat log.

```
01  import java.io.IOException;
02  import java.io.PrintWriter;
03  import javax.servlet.ServletContext;
04  import javax.servlet.ServletException;
05  import javax.servlet.http.HttpServlet;
06  import javax.servlet.http.HttpServletRequest;
07  import javax.servlet.http.HttpServletResponse;
08
09  public class ServletDebugging extends HttpServlet {
10
11      @Override
12      protected void doGet(HttpServletRequest request, HttpServletResponse response)
13              throws ServletException, IOException {
14
15          // parameter "name"
16          String strpm = request.getParameter("name");
17
18          ServletContext context = getServletContext( );
19
20          // checks if the parameter is set or not
21          if (strpm == null || strpm.equals(""))
22          context.log("No message received:", new IllegalStateException("Sorry, the
23          parameter is missing."));
24          else
25          context.log("Here is the visitor's message: " +strpm);
26
27      }
28  }
```

```
12-Nov-2014 15:13:10.902 SEVERE [http-nio-8084-exec-45] org.apache.catalina.core.ApplicationContext
.log No message received:
 java.lang.IllegalStateException: Sorry, the parameter is missing.
```

Figure 19: Output as visible in Apache Tomcat log

9. Internationalization

For building a global website, some important points are considered which includes language related to user's nationality. Internationalization isenabling a website for providing content translated in different languages according to user's nationality.

9.1 Methods

For finding visitors local region and language, these methods are used:

Method	Description
`String getCountry()`	Returns the country code.
`String getDisplayCountry()`	Returns a name for the visitors' country.
`String getLanguage()`	Returns the language code.
`String getDisplayLanguage()`	Returns a name for the visitors' language.
`String getISO3Country()`	Returns a three-letter abbreviation for the visitors country.
`String getISO3Language()`	Returns a three-letter abbreviation for the visitors language.

9.2 Example

The example displays the current locale of a user. Following project is created in NetBeansIDE:

```
Project Name: WebApplicationInternationalization
Project Location: C:\Users\Test\Documents\NetBeansProjects
Servlet: ServletLocale
URL Pattern: /ServletLocale
```

Listing 22: ServletLocale.java

```java
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Locale;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class ServletLocale extends HttpServlet {

@Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
           //Get the client's Locale
      Locale newloc = request.getLocale();
      String country = newloc.getCountry();

      // Set response content type
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      // this sets the page title and body content
      String title = "Finding Locale of current user";
      String docType =
      "<!doctype html public \"-//w3c//dtd html 4.0 " +
      "transitional//en\">\n";
       out.println(docType +
        "<html>\n" +
        "<head><title>" + title + "</title></head>\n" +
        "<body bgcolor=\"#C0C0C0\">\n" + "<h3>" + country + "</h3>\n" +
        "</body></html>");
    }
}
```

Listing23: index.html with location hyperlink as URL-pattern – ServletLocale

```html
<html>
<head>
<title>User's Location</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
    <p>Click on the following link for finding the locale of visitor:</p>
    <a href="ServletLocale"><b>Location</b></a>
</body>
</html>
```

Listing24: web.xml with URL-pattern as /ServletLocale

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchem
<servlet>
    <servlet-name>ServletLocale</servlet-name>
    <servlet-class>ServletLocale</servlet-class>
</servlet>
<servlet-mapping>
     <servlet-name>ServletLocale</servlet-name>
     <url-pattern>/ServletLocale</url-pattern>
</servlet-mapping>
<session-config>
<session-timeout>
30
</session-timeout>
</session-config>
</web-app>
```

Figure 20: Displaying index.html